

Competing Explanations, Persisting Rules: Grounded Defense against Misleading Charts in Executing Web Agents

Anonymous submission
Paper ID: NAACL-2027-XXXX

Abstract

Vision–language agents that execute multi-step web tasks treat charts as evidence: the way a chart is read enters tool calls, submissions, and actions. Charts, however, admit readings that are both faithful to the rendered image and wrong about the underlying data, because correct visual observation and misleading encoding conventions coexist in the same figure. Existing defenses operate at the level of single question–answer rounds and do not address how an interpretation, once revised, remains in force over a stateful execution. We present CERS (Competing-Explanation Rule State), a harness-level defense that organizes chart interpretation around two coupled mechanisms: a competing-explanation chain that expresses the current judgment as $O \wedge B \Rightarrow C_q(a)$ —an observation, an interpretation rule, and the conclusion an action requires—and a persistent rule state that records every validated rule revision with its supporting evidence and scope, and re-uses it in later reasoning. On a paired static panel of 140 executable web tasks built on MIS-VISAGENTBENCH, CERS improves the robust accuracy of a 27B open-weight VLM agent from 57.9% to 68.6% in the original round and replicates the gain in an independent round (52.1% to 64.3%; McNemar $p = 0.0025$), with a smaller gain on an 8B model (46.4% to 57.1% for the most compatible variant). In stateful online execution over a closed deployment of twelve live tasks spanning both chart variants, the ordinary agent submits six wrong actions across the 27B batches while the CERS arm submits none in any batch or chart variant; in the frontier-model batches the defense arm converts correct misleading-chart decisions into recorded abstentions (exact sign test $p = 0.002$) rather than wrong submissions, and a stability rerun agrees with the formal run on 37 of 48 unit outcomes, with every disagreement an abstention–correct flip.

1 Introduction

Large language model agents act by looping through observation, reasoning, and action in an external environment; what the agent believes about the information it reads enters tool calls and operations directly, making the reliable interpretation of external content a precondition for execution safety. Prior work has shown that indirect prompt injections buried in web pages and tool returns can steer an agent away from the user’s intent (Zhan et al., 2024; Evtimov et al., 2025; Johnson et al., 2025), that perturbations of visual input can hijack multi-step web agents (Wu et al., 2024), and that deceptive interface design can be reproduced and probed systematically (Wang et al., 2025). These findings indicate that an environment does not only intervene on an agent through explicit instructions; it can also act on the way the agent understands and uses information, and thereby shape the decisions the agent makes autonomously.

Charts are a prominent part of such external information: they organize data into visual forms that support comparison and inference, and lend an appearance of evidential support to whatever judgment is attached to them. Yet correct and misleading readings can coexist in the same figure: charts support misleading inferences even when drawn faithfully (Lisnic et al., 2023; Nadib et al., 2026). A curve moving upward is observed accurately, but whether it represents growth depends on the axis mapping currently in force; color intensity and bar height are likewise read through the present figure’s encoding relations, and a misleading presentation can induce an agent to apply its customary reading rules unchanged, converting visible evidence into a wrong conclusion (Figure 1).

Chen et al. (2025) document that multimodal models are systematically vulnerable to such manipulations in chart question answering, (Mahbub et al., 2025, 2026) trace the vulnerability and its par-

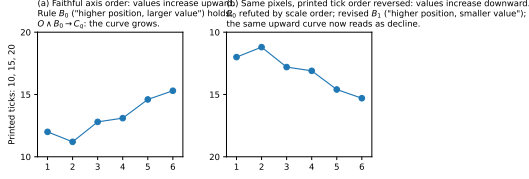


Figure 1: A schematic instance of the phenomenon. The two panels show the same drawn series under two printed axis orders; the customary reading rule holds in (a) and is refuted by the scale order in (b), where the same upward curve supports the opposite trend judgment. The benchmark pairs each task with such variants; model inputs never reveal which panel is which.

tial mitigation across model families, and our companion work under review formalizes the execution-level consequence: on a paired benchmark of executable web tasks, biases induced by misleading charts propagate into subsequent actions and change task outcomes.¹

Against misleading charts, model-level defenses have improved chart question answering through tabular answering and re-plotting procedures (Ton-glet et al., 2026), but the reliability of the verification itself remains open: intrinsic self-correction can damage initially correct answers (Huang et al., 2023; Li, 2025), and when the evaluating component is itself exposed to the environment, reflection and search can open new attack surfaces (Wu et al., 2024). Chart-domain studies observe a further gap: a model may correctly identify chart properties during extraction and still fail to let them govern downstream inference (Mahbub et al., 2026). These observations point to two connected requirements: correction must rest on evidence that can be checked, and a corrected interpretation must actually change subsequent judgments—repeated observation or an additional review round can as easily re-adopt an inapplicable reading as discover an error.

Stateful execution raises requirements beyond chart question answering: an understanding formed from one chart feeds later planning and operations even after the chart has left the viewport, so a verified axis mapping should keep governing later judgments, yet become revisable when sufficient new evidence appears. Reflexion-style memory carries verbalized feedback into later attempts (Shinn et al.,

2023), and recent agent-memory systems organize experience into retrievable state (Wang et al., 2024; Hu et al., 2025); an inductive chart requires something more specific, the current validity of interpretations: which rules have evidential support, which have been refuted, and what new evidence suffices to change an accepted reading? We pose one unified question: *how can an agent reliably revise visual interpretation rules, and keep validated rules in force over subsequent execution?*

We answer this question with a defense that combines *competing explanation chains* with a *persistent rule state*. When the agent forms a chart-dependent judgment, the harness expresses it as $O \wedge B \Rightarrow C_q(a)$: an observation O in the chart, an interpretation rule B that turns the observation into task meaning, and the conclusion $C_q(a)$ that an action a requires under the task q . The harness then builds candidate chains with different interpretive bases on the same complete chart, turning default visual conventions into explicit objects of test, and compares their applicability using evidence drawn from the figure itself. Validated rule revisions enter the agent’s runtime harness as an interpretation state that later reasoning reads and updates by scope; subsequent decisions reuse the rules currently in force, and re-enter verification only when new effective evidence challenges them. Competing explanations determine why an interpretation needs to change; the persistent state carries that evidenced change into later actions.

Contributions: (1) a correction framework based on competing explanation chains, in which the relation $O \wedge B \Rightarrow C_q(a)$ unifies visual observation, interpretation rules, and action judgments, and customary readings are tested against in-chart evidence so the rules responsible for wrong inferences are identified and revised; (2) a stateful defense for continuing execution, in which validated rules, revision evidence, and scope conditions are maintained as task-scoped harness state, so later decisions reuse confirmed chart readings and revise them only on new effective evidence.

2 Related Work

Research on misleading visualizations has developed along two lines that our defense connects. The first line studies how charts mislead human readers: Lisnic et al. (2023) show that deception relies less on outright visual distortion than on faithful drawings combined with selective context and framing

¹Companion submission under review; the paired benchmark MISVISAGENTBENCH and the execution-level evaluation protocol it introduces are described where first used in this paper.

(Nadib et al., 2026). The second measures the same phenomenon in models: benchmarks evaluate multimodal LLMs on misleading chart QA (Chen et al., 2025; Bharti et al., 2024; Alexander et al., 2024); Mahbub et al. (2025) and Mahbub et al. (2026) quantify how truncated axes, distorted aspect ratios, and manipulated encodings degrade chart reasoning and probe mitigations; and recent studies ask whether agents recognize misleadingness unprompted (Panda, 2026; Blasilli and Angelini, 2026; Seto et al., 2026). Method-side responses include tool-augmented chart reasoning (Kaur et al., 2026), structured extraction pipelines (Das et al., 2025), misleading-chart detection (Tonglet et al., 2025), and agentic dual-path frameworks that separate data-fidelity from conclusion checking (Zhang et al., 2026). Tonglet et al. (2026) provide the closest model-level defense, protecting multimodal LLMs through tabular answering and chart re-plotting; these procedures repair a single question–answer round, while CERS targets the interpretation rules that recur across the rounds of an executing agent and maintains their validated revisions in runtime state.

A second line concerns agent-environment integrity. Indirect prompt injection benchmarks show that tool and web content can steer agents (Zhan et al., 2024); web-agent security benchmarks measure targeted hijacking (Evtimov et al., 2025); injection reaches the very representations agents perceive, such as HTML accessibility trees (Johnson et al., 2025); automated red-teaming generates such attacks at scale (Wang et al., 2025) and runtime frameworks intercept them (Zhao et al., 2026). Interface-level defenses separate instructions from data (Chen et al., 2024a) or train against injected content (Chen et al., 2024b). Wu et al. (2024) dissect the robustness of multimodal agents as compound systems and show that reflection and tree search, the components commonly added for self-correction, open new vulnerabilities when the evaluator itself is exposed to attacked content. Our defense differs in what it protects: not the agent’s instruction hierarchy but the interpretation layer between visual observation and task judgment, where a misleading chart exerts its effect without any injected instruction.

The third line concerns persistent state and correction. Verbal reinforcement carries textual feedback across attempts (Shinn et al., 2023), self-consistency aggregates reasoning paths (Wang et al., 2022), and intrinsic self-correction with-

out external feedback degrades reasoning (Huang et al., 2023; Li, 2025). Agent surveys organize the design space of state and memory components (Wang et al., 2024; Hu et al., 2025), and Koh et al. (2024) establish that realistic visual web tasks remain hard enough that single-decision correctness cannot be taken for granted. CERS borrows the state-carrying idea but changes the carried content: the state records interpretation rules with evidential support, applicability scope, and revision history, so later steps inherit what was verified about the chart rather than a verdict about the answer.

3 Method

CERS rests on one object: the interpretation rule B that connects visual observation to task judgment. Competing explanation chains make these rules explicit and test them; persistent state organizes their revision evidence, current validity, and applicability scope, so that they participate in later interaction. This section develops the two mechanisms in turn after fixing the task setting.

3.1 Task Modeling and Overall Framework

Consider an agent executing a task q : at step t it receives an observation o_t and, given history H_t , proposes $\tilde{a}_t$. When the proposal depends on chart understanding, let I_t denote the complete chart observation relevant to the current judgment—or the chart observation actually obtained earlier in the task when the chart has left the current page. We express a chart-driven interpretation as

$$O \wedge B \Rightarrow C_q(a), \quad (1)$$

where O is an observation in the chart, B is the interpretation rule required to convert the observation into task semantics, and $C_q(a)$ is the proposition that the action, or high-level choice, a meets the requirements of task q . The representation separates a visible phenomenon from the condition under which it is interpretable: the observation that a curve moves upward, jointly with the rule that in the current axis a higher position means a larger value, supports the conclusion that the interval grows and the action is admissible; a distorted bar chart challenges the rule that bar-height ordering preserves value ordering, and a mismatch against verified numeric relations refutes precisely that rule’s applicability. Both O and B may carry multiple items, expressed in structured natural language covering entity binding, numeric relations,

encoding direction, time range, and comparison conditions. Relations that can be computed explicitly, such as numeric comparisons and orderings, are checked with program assistance; the remaining semantic judgments are verified jointly with the public task requirements.

The method maintains a within-task rule state M_t^B and, for chart-dependent judgments, runs the process *current interpretation* $\rightarrow$ *competing-explanation generation* $\rightarrow$ *in-chart applicability verification* $\rightarrow$ *rule-state update* $\rightarrow$ *action re-derivation*; later steps read the applicable rules from the current context and restart competing verification only when a relevant judgment is first formed, when new evidence conflicts with an existing rule, or when a rule’s applicability conditions are in question. Local verification decides whether a rule deserves acceptance or revision; within-task maintenance decides in which later steps it remains effective. The underlying model’s parameters stay frozen, and the rule state evolves only with observations and verification results actually obtained.

3.2 Competing Explanation Chains

3.2.1 Making the Current Explanation Explicit

When the agent proposes a chart-dependent choice $\tilde{a}_t$, the interpretation module forms, from the complete chart, the public task, the admissible history, and the current rule state,

$$\pi_0 = \langle O_0, B_0, p_0 \rangle, \quad p_0 = C_q(\tilde{a}_t), \quad (2)$$

stating separately which phenomena the chart is claimed to contain, which rules carry the observation to the conclusion, and which evidence those rules hold or lack; where the current choice lacks support, the module records an interpretation gap directly. “The color is darker” stands as an observation, while “the risk is higher” stands only under a color-to-risk mapping: making the chain explicit turns such omitted conditions into objects for verification and directs the counter-questions that follow.

3.2.2 Counter-Questions and Competing Explanations

The competing module poses three connected questions around the current chain: whether the current observation suffices for the conclusion, which interpretation rules the inference depends on, and whether the same chart admits another evidence-

based explanation under which the current conclusion fails, or under which the current action continues to hold on different grounds. From these questions the module generates a bounded candidate set

$$\Pi_t = \{\pi_0, \pi_1, \dots, \pi_K\}, \quad \pi_i = \langle O_i, B_i, p_i \rangle, \quad (3)$$

where each conclusion p_i may support the current action, refute it, or support a different task judgment. All candidates refer to the same complete chart, and the counter-questions act on the interpretive conditions between observation and conclusion. A competing chain is meaningful only if it differs substantially in observation correspondence, interpretation rule, applicability condition, or scope of derivation: two chains that both accept “the curve rises” but adopt forward and reversed value mappings disagree precisely on the applicability of B . A reversed interpretation enters as a conditional hypothesis and stays unverified until the chart’s scales support its condition; the module may also return insufficient support and is not required to produce an opposing conclusion for every task.

3.2.3 Normalization and Shared Rules

Generated chains are normalized over observation, rule, applicability scope, conclusion, and evidence dependence. Explanations that differ only in wording are merged, and a rule used in several chains shares one verification state; different entities or comparison relations keep their separate evidence. Whereas self-consistency aggregates a distribution of answers over repeated reasoning paths (Wang et al., 2022), the object of verification here is the rule each path depends on and the evidence for it: repeated generations of the same rule add no evidential weight, and a new explanation changes later verification only by contributing different evidence or exposing a new applicability problem. In the deployed pipeline the candidate set of a verification node is the frozen per-task pool of one to three pre-generated O/B records (275 records across the panel), so the chain budget of Eq. (3) is instantiated as K equal to the pool size and diversity is a property of pool construction—candidate rules differ in object, condition, or scope by design; the online arm re-derives chains at runtime under the same budget. This multi-candidate structure relates to multiagent debate (Du et al., 2023; Liang et al., 2024) and to single-path self-refinement (Madaan et al., 2023), but the competitors disagree only over

the interpretive bases of one rendered chart, and the adjudicator is an applicability test against that chart rather than a vote or an LLM-judged winner.

3.3 In-Chart Applicability Verification

Verification first extracts the disputes that genuinely affect conclusions among the candidates, concentrating on the disputed mapping when chains agree on the observation, and on observation binding before rule applicability when the dispute concerns entity–value correspondence. For each chain the module outputs

$$V_i = (v_i^O, v_i^B, v_i^{\Rightarrow}, E_i), \quad (4)$$

where each verification status takes the values support, refute, or undetermined, and E_i records the in-chart evidence. Observation O_i is checked for presence in the chart and for the correctness of entity, value, time, and unit binding; rule B_i is checked for support by the current chart and satisfaction of its applicability conditions; the derivation $O_i \wedge B_i \Rightarrow p_i$ is checked for sufficiency of the confirmed premises, including candidate coverage, range, and task conditions. Verification always proceeds against the complete chart, and the evidence record contains the actual observation identifier, the content read, and which rule the content supports or refutes. The deployed checklist operationalizes the three statuses: for O , located support or a genuine contradiction in the figure, with no inserted quantifiers or causal claims; for B , explicit tests of the rule’s necessary conditions against public facts, including whether the known labeled references would have the relations a proposed decoding predicts and an explicit order comparison before any common scale is endorsed; for the derivation, candidate coverage, range, and task-scope conditions must be confirmed premises. Verification takes the normalized dispute propositions as input; the source role or repetition count of a candidate is never a credibility signal, and observations and interpretive conditions are confirmed before conclusions, so a preferred answer cannot justify a rule retroactively.

For rules that are mutually incompatible over the same objects and scope, the method checks which in-chart information can discriminate them: if two explanations each retain unresolved evidence, the dispute is kept open; if they apply to different axes, series, or ranges, the applicability conditions are split rather than forced to a single choice. Verification may support a replacement rule, or only

negate the original one: discovering that bar height and visible value ordering disagree does not entail that “shorter means larger,” and the sound revision may instead be that bar height in the current figure does not directly support quantity ordering, which must rely on verified entity–value relations. Rule revision therefore includes both replacing a mapping and disqualifying an inference route within the current chart.

3.4 Rule Revision and Persistent State Update

Each interpretation rule is maintained as a record

$$m_i = \langle \text{id}_i, B_i, \sigma_i, s_i, E_i, \Delta_i, v_i \rangle, \quad (5)$$

where id_i identifies the rule for later reference, B_i is the current interpretation rule, σ_i is its applicability scope over chart objects, encoding components, and semantic conditions, s_i is its status among pending, currently active, disputed, and revoked, E_i is the observed evidence supporting or refuting the rule, Δ_i links the original rule, the revised rule, and the reason for the change, and v_i is the revision version. The state stores how the current chart is to be read and on what evidence; object choices, answers, and click actions are produced by the task reasoning itself and recorded in the execution history. A rule’s scope binds by the conditions under which it holds—a position–value mapping to the numeric axis and series concerned, a color reading to the current legend and metric, an overall-trend rule to temporal order and comparison range; page changes unrelated to the task goal do not invalidate a rule by themselves.

Verification produces rule-revision proposals Δ_i^B , applied as $M_{t+1}^B = U(M_t^B, \Delta_i^B)$. If the original rule is supported and the proposal lacks effective evidence, the rule is kept and the proposal is retained as pending; if the original is refuted with a supported replacement, the replacement activates in the affected scope; if the original is refuted without a settled replacement, the rule is revoked and the interpretation gap recorded; if a competing rule remains in effective conflict, the affected rules enter a disputed status while verification continues within budget; and rules that apply to different objects or scopes are stored separately rather than overwritten globally. The “currently active” status expresses the method’s judgment under the evidence at hand; new effective evidence can change it, and can also show that an earlier verification was itself mistaken. After revocation, content, scope,

and reason remain in the revision history; if the same rule reappears, it must answer the recorded refutation or supply evidence that changes the applicability judgment—repetition, later position, or a more confident phrasing forms no new revision ground by itself.

A revision record answers why the original rule had to change, what interpretation is adopted now, and under which conditions the change applies: for a reversed linear axis, the record replaces “higher position, larger value” because the visible scale increases downward, states that an upward move of the bound series now reads as a decrease, and scopes this to the current chart, axis, and series—supporting trend judgments over further intervals without storing a fixed answer of growth or decline.

An invalidated rule does not by itself condemn the current action, since $\neg B_0 \not\Rightarrow \neg C_q(a_0)$. After updating B , the method re-examines the current judgments that depended on it: a judgment that lost its basis enters re-review and may be retained if other effective interpretations still support it, so the two admissible outcomes are a rule change with the action retained (a wrong reason for a right choice) or rule and action changing together. Judgments already recorded in the history are marked as basis-changed for later re-confirmation; the original interaction history stays intact, and the currently effective rules are organized by a separate read interface.

3.5 Reading and Using Rules over Continued Execution

Let the current interpretation context c_t collect the relevant chart objects, encoding components, metrics, and task scope. The state reader returns $\mathcal{R}_t = \text{Read}(M_t^B, c_t)$, of which the directly usable part is

$$\mathcal{R}_t^{\text{active}} = \{m_i \in M_t^B : s_i = \text{active} \wedge \text{Match}(\sigma_i, c_t) = \text{true}\} \quad (6)$$

Formally, $\text{Match}(\sigma_i, c_t)$ evaluates the rule’s object, encoding component, metric, and time window against the current interpretation context and returns applicability, non-applicability, or undetermined; an undetermined match triggers verification of the corresponding condition rather than a default assumption that a stored rule covers the new chart. Disputed rules never enter $\mathcal{R}_t^{\text{active}}$: they surface only as re-verification evidence in the read result, so a dispute can compel a new verification round but cannot be used as a current fact. The read re-

sult also carries the disputes and revocation records relevant to the current judgment, so the agent can separate interpretations still in force from ones that appeared before and lost support; revoked content never participates in derivation as a current fact. When a chart has temporarily left the viewport, the reader keeps supplying applicable rules through the chart references established within the task, and new chart objects or encoding changes are bound and verified afresh.

The base agent produces its next proposal from the current observation, history, and rule view,

$$(\tilde{a}_t, \tilde{\pi}_t) = \mathcal{F}_\theta(q, o_t, H_t, \mathcal{R}_t), \quad (7)$$

where $\tilde{\pi}_t$ is a short interpretation record of the present judgment. Rules about the same chart can remain valid while the correct action changes with task conditions, so the method reuses interpretation evidence across steps while re-deriving actions from the current observation and the public task. A proposal that depends on a rule revoked within its scope is returned the revision evidence and required to re-derive; a proposal bringing evidence sufficient to challenge an active rule re-enters competing verification. This restrains old conventions from regaining force without grounds while preserving a channel for correcting earlier verification mistakes. Ordinary actions that involve no change of chart interpretation follow the original interface; the persistent state acts through later contextual reads, with complete competing verification only when a trigger holds.

Three observable entry points start verification: the first chart-dependent judgment of a task (which expands the current interpretation and establishes the task’s rule state), an interpretation conflict (a later proposal uses a rule inconsistent with the effective state, or an existing rule is challenged with in-chart evidence), and an applicability change (the chart, its encoding, the series binding, or the task scope changes in ways that can affect interpretation). Entry points are determined by actual observations, task context, and action proposals; task correctness and hidden scoring never participate in triggering.

After a rule update, the agent re-judges the action and records *Keep* (the choice remains supported by an effective interpretation and the task conditions), *Revise* (relevant evidence negates the choice and an alternative gains the support the task requires), or *Unresolved* (an interpretive dispute that could

still change the judgment persists, or necessary evidence is missing). Showing that a candidate exceeds the current choice refutes “the current choice is highest,” for example, while confirming the replacement still requires the comparison the public task demands; such checks belong to task-semantic verification, not to one fixed comparison formula. The unresolved status continues whole-chart verification within the remaining budget, and when the budget is exhausted the unfinished reason is recorded explicitly. Verification suggestions, actual choice revisions, and final submissions are recorded separately, and only operations carried out by the executor constitute action outcomes.

3.6 Full Algorithm and Integration

Algorithm 1 (Appendix A) summarizes one execution of CERS under the candidate budget K and the review budget L for a single verification node. The framework enters an existing agent at three points—the observation entrance establishes chart references, the pre-execution point performs verification and rule revision, and the later-reasoning entrance reads the currently applicable rules; planning, execution, and review modules share the rule state within one task, while action capabilities and the environment interface follow the original system. Explanation generation and visual verification can be carried out by different calls of the same frozen model. Deterministic code handles structured record keeping, management of normalized merges, version updates, scope retrieval, and checks of relations that can be computed explicitly. The competing-chain count, review rounds, and trigger settings are fixed before a run, and the extra calls and context overhead are charged to the method’s cost. The rule state is isolated per task: the runtime uses only the chart and interaction information the task permits, while the paired chart, hidden answers, and scoring feedback stay on the evaluation side. Charts, rules, and action records are stored separately so that later analysis can trace how evidence changed and whether operations followed.

4 Experiments

This section reports the two evaluation deployments of CERS: a static paired panel that isolates the verification core, and a stateful online deployment that adds the persistent rule state inside a live browser harness.

4.1 Setting

The task basis is inherited from our companion benchmark MISVISAGENTBENCH, which pairs 140 executable web tasks—47 business, 35 environment, 19 health, 39 public-statistics, in 21 mechanism families—with a faithful chart and a misleading chart whose presentation violates an encoding convention while the underlying data remain drawable.² Model inputs never contain gold labels, mechanism labels, the other chart variant, or scoring feedback.

Static panel. For every task the agent sees the benchmark’s misleading chart—scored against its frozen original label—plus the public goal and four options, and must submit a selection; no browser navigation occurs. Six configurations are compared on identical inputs and decoding (temperature 0.7, top- p 0.8, top- k 20, fixed seed, thinking disabled): direct selection and five READ–VERIFY–DECIDE configurations (Section 3) with frozen prompts and three requests per task, whose candidate chains are the frozen per-task pool of one to three pre-generated O/B records (275 across the panel). V3 verifies O and B in distinct schema-typed passes, V4 fixes explicit numeric references, V5 (full) adds same-series query-point constraints with scale, time-window, and policy-boundary checks, V6 checks each necessary condition of B item by item, and V7 keeps V4’s prompts with an abstention-aware decision repair phase; full prompts are in the supplementary material. Interface failures and empty selections remain in the 140 denominator; *robust accuracy* throughout is the fraction of tasks whose selection matches the frozen original label, the same full-denominator quantity for every configuration. The supplementary material contains the frozen prompts, typed output schemas, per-request decision logs with seeds and usage, the interface contract, and the scoring code, so every number in this section can be recomputed from the released artifacts once the companion benchmark’s data license permits public release.

Online stateful deployment. Twelve tasks spanning all four domains (six business, three environment, two health, one public-statistics, taken in fixed identifier order from the benchmark’s task lists before execution) run as live browser sessions under a closed deployment: for each task,

²Companion submission under review; instance construction, taxonomy, and scoring are defined there.

Config	Target	Rate	Δ	Other	Empty	IF	27B replication				8B cro		
						Config	Tar.	Δ	p	95% CI	Tar.	Δ	p
Direct	81	57.9%	—	59	0	Direct	73	—	—	—	65	—	—
V3	74	52.9%	−7	60	0	V3	75	+2	<.001	[+.09,+ .22]	50	−15	.18
V4	87	62.1%	+6	53	0	V4	86	+13	.0025	[+.05,+ .23]	77	+12	.07
V5 (full)	96	68.6%	+15	44	0	V5	90	+17	.0013	[+.06,+ .23]	75	+10	.04
V6	84	60.0%	+3	56	0	V6	86	+13	<.001	[+.07,+ .22]	71	+6	.23
V7	91	65.0%	+10	49	0	V7	89	+16	.0025	[+.05,+ .23]	80	+15	.02

Table 1: Original round on the 27B open-weight VLM

Table 1: Original round on the 27B open-weight VLM (140 tasks; temperature 0.7, top- p 0.8, top- k 20, fixed seed). “Target” counts selections matching the original label; Δ is the net change against Direct; “Other” counts wrong selections; “IF” counts interface failures, which remain in the denominator. The 24-task development subset is scored separately and reported in Appendix B.

both chart variants (official misleading chart, clean chart) execute under both systems—ordinary execution without defense, and the CERS arm whose hooks, rule state, and re-derivation loops run inside the harness with the actor driving a real browser. The submission interface is a strict typed protocol (typed_scalars_v2; at most eight actor calls, one re-verification round, temperature 0), in which a unit ends as a scored submission, an abstention with a recorded reason, or a failure. Three actor systems are used: the 27B model under strict response formats (Batch A), the same model behind a schema-guided proxy mirroring the sealed structural rules (Batch B), and the frontier model of the pilot reached directly through the gateway (Batch C, the formal result, plus one stability rerun).

4.2 Static Panel Results

Table 1 reports the original round under the 27B open-weight VLM: direct selection sits at 57.9% (81/140), and the full configuration V5 reaches 68.6% (96/140), a net gain of 15 decision points (10.7 points of accuracy) with no increase in abstention or interface failure. Because the panel is entirely single-decision-point, that gain cannot be attributed to information gained after selection: verification changes the decision that was already available, which is the quantity the formulation of Section 3.1 isolates.

Table 3 reports a matched-configuration replication designed to separate sampling variance from configuration effects: direct selection and the full configuration V5 are each re-executed under three request seeds (12345, 22345, 32345) on a single-GPU deployment whose engine configura-

Table 2: Replication on the same 27B model (left; 8.84M tokens, 2,230 requests) and transfer to an 8B open-weight VLM (right; 8.14M tokens, 2,184 requests). Δ is the paired net change against the Direct configuration of the same run; p and the exact 95% CI come from two-sided exact McNemar tests on the wrong-target discordant pairs (McNEMAR, 1947), which exclude abstentions and interface failures; Δ retains them in the denominator.

Arm	Requests	Selection acc.	Δ	p
Direct (3 seeds)	3/task	56.4%	—	—
SC3 (majority)	0 extra	53.6%	−3	.375
Tabular (bounded)	2/task	28.6%	−3	.51
CERS V5 (3 seeds)	12/task	64.3%	+17	.0013
Clean-chart control	1/task	82.1%	+33	<.001

Table 3: Matched-configuration replication (Appendix B): three request seeds for Direct and CERS, SC3 majority over the Direct seeds at no extra requests, the bounded tabular-answering baseline (two requests per task; 55 abstentions and 18 transcription interface failures included in the denominator), and the clean-chart control. Δ and p are paired against the Direct seed-12345 run; accuracy is the full-denominator fraction of tasks whose selection matches the frozen original label.

tion matches the frozen service except for batch admission, so all comparisons are internal to that deployment (Appendix B records the protocol). The same run supplies three baselines at no additional engine cost: a self-consistency vote SC3 over the three direct-selection seeds (Wang et al., 2022) (majority among non-null selections, ties recorded as abstentions), a tabular-answering baseline in the style of Tonglet et al. (Tonglet et al., 2026) (bounded transcription of the comparison-relevant chart rows, then a decision taken from the table alone with the image withheld), and a clean-chart control that runs direct selection on the paired clean charts.

Table 2 (left) reports the single-run replication

that keeps model, seeds, and prompts frozen: direct selection drops from 81 to 73 of 140 under an identical configuration, quantifying the sampling variance of agentic decoding itself, so every comparison must be made within the same run. Within-run, V5 gains +17 paired decisions ($p = 0.0013$; 95% CI [+0.061, +0.231]), V4, V6, and V7 gain +13, +13, +16 ($p \leq 0.0025$ each), and V3 returns to +2 despite strongly directional wrong-target flips (30 against 6, $p < 0.001$) because its interface failures absorb the difference—the separation between decision quality and interface robustness that Section 4.3 examines in deployment. Paired transfer counts are one-directional in aggregate (57 keep, 33 error-to-target, 11 target-to-error, 5 target-to-empty, 0 empty-to-target), and the panel remains abstention-free outside V3’s interface failures.

Baselines and controls. Three observations bound what the verification gain can be attributed to. First, the clean-chart control: direct selection rises from 56.4% on misleading charts to 82.1% on the paired clean charts (115/140; paired net +33 against its own misleading-chart run, $p < 0.001$, exact CI [+0.16, +0.27]), which quantifies the execution-level damage the misleading presentations cause and the room any defense has to recover. Second, the majority-vote baseline: SC3 reaches 53.6% (75/140), statistically indistinguishable from direct selection (paired net -3 , $p = 0.375$); aggregating diverse decoding paths neither recovers the misleading-chart losses nor approaches the verification gain, consistent with the argument of Section 3.1 that the object of correction is the interpretation rule, not the answer distribution. Third, the tabular-answering baseline abstains on 55 of 140 units (its bounded table does not carry the task-relevant evidence) and records 18 transcription interface failures; on the 67 units where it selects, submission precision is 59.7% (40/67), and its paired net against direct selection is -3 ($p = 0.51$). Generic mitigation therefore neither transfers the tabular-answering gains reported for chart QA to execution tasks nor exploits the within-chart evidence that CERS uses; the seed-level variance of the two arms is reported with Table 3 once the three-seed replication completes (mean $\pm$ sd over seeds for both arms, pooled McNemar across seeds).

Cross-model transfer. On the 8B open-weight VLM, direct selection drops to 46.4% and the picture splits (Table 2, right): V7 gains +15 ($p = 0.021$; CI [+0.017, +0.210]), V5 gains +10

($p = 0.044$), V4 +12 and V6 +6 remain within noise, and V3 collapses—51 of 140 runs end in interface failures of the schema-typed protocol, against 8 and 6 in the other rounds, for a net of -15 despite directional wrong-target flips (18 against 10). The interaction is instructive: the same separated verification prompts that produce directional wrong-target flips on the 27B model overwhelm the 8B actor. Interface robustness is thus a first-class variable, and prompt content and interface contract must be evaluated jointly rather than as one “method” factor.

Where the gains concentrate. Stratified by mechanism family, V5’s net gains concentrate where misleading power comes from scope and reference manipulation: the subset-extrapolation family (22 tasks) yields +18 and +11 across the two rounds, while non-linear-axis and dual-axis families are neutral to -6 . By domain, business carries the largest share (+16 and +4), public statistics is neutral then positive (0 and +9), health is +6 in both, and the environment domain is the only negative one in both rounds (-5 , -9). The development subset contributes one decision point to each headline gain, and costs remain bounded: 8.84M and 8.14M tokens across the replication rounds (2,230 and 2,184 requests) against 8.86M in the original round, a $2.6\times$ request overhead over direct selection with frozen prompts and no parameter updates.

4.3 Online Stateful Results

A pilot (6 tasks, frontier model, 12 units) already showed the qualitative pattern (5 correct selections for the ordinary arm against 1 for CERS). Because the deployment’s submission protocol is fail-closed, the primary online metric is the *wrong-submission rate*: across the four batches, ordinary execution produced six wrong submissions, all under the 27B actor on misleading charts, while the CERS arm produced none in any batch or chart variant (0 of 96 defense-arm units). Aggregate accuracy moves in the opposite direction: ordinary execution loses 3–5 decision points on misleading charts relative to its own clean-chart performance (8 vs. 10 in the formal batch; 7 vs. 10 and 7 vs. 12 in the 27B batches), and the defense arm converts correct ordinary decisions into recorded abstentions rather than corrections. The paired task-level effect is strictly one-directional in both frontier-model batches (ordinary preserves 10 and 13 decisions that the de-

Unit class	Cor.	Abst.	Bug.	Fail.	Verification budget.
------------	------	-------	------	-------	----------------------

Batch C (formal run)					Stability. Re-running Batch C under the identical
Ordinary $\times$ misleading	8	3	–	1	configuration changes 11 of 48 unit outcomes, ev-
Ordinary $\times$ clean	10	2	–	–	ery one of them a flip between a correct submission
CERS $\times$ misleading	2	7	1	2	and an abstention (agreement 37/48); no wrong
CERS $\times$ clean	6	4	–	2	submission appears in either run, so the “never
Batch C rerun (stability)					submit a wrong answer” property is stable while
Ordinary $\times$ misleading	10	1	–	1	completion throughput is not. Token budgets are
Ordinary $\times$ clean	10	2	–	–	comparable across the two runs (1.80M input and
CERS $\times$ misleading	2	5	–	5	129K output tokens, 322 attempts and 242 browser
CERS $\times$ clean	5	4	–	3	operations, versus 1.81M and 115K, 331 and 254),

Table 4: Online stateful deployment: 12 tasks spanning four domains, each run on the misleading chart and its clean pair under ordinary execution and the CERS arm (48 units per batch). “Cor.” counts submitted actions matching the original label; “Abst.” recorded abstentions; “Bug.” re-verification budget exhaustion; “Fail.” engineering and response-format failures (including actor-call-limit terminations). All submissions across both batches are correct (26/26 and 27/27).

fense arm turns into abstentions; the reverse never occurs; exact sign tests $p = 0.002$ and $p < 0.001$ on discordant pairs), and submission precision is 100% for both systems in those batches (26/26 and 27/27).

Interface decomposition. The three actor systems separate structural from semantic failure modes. Under the strict interface without guidance (Batch A), 22 of 24 defense-arm units die in engineering failures, and the ordinary arm ends in two call-limit exhaustions on misleading charts. A schema-guided proxy mirroring the sealed structural rules (Batch B) drives the ordinary actor’s structural failures to zero (12/12 on clean charts) but cuts defense-arm engineering failures only modestly (21 of 24): the binding constraint is semantic, since exact public-leaf values, bounded summaries, and the question–response count coupling cannot be expressed by any response format—components added for self-correction become the exposed surface, as Wu et al. (2024) analyze for compound systems. Running the frontier model directly (Batch C) removes the structural interface almost entirely (defense-arm engineering failures 4 of 24), and the remaining mass is honest abstention—12 unresolved_public_evidence verdicts, mostly on misleading charts, where verification cannot close the gap between the printed chart and the question’s required scope within the re-

5 Conclusion

We treated misleading charts as a runtime evidence problem for executing web agents rather than a benchmark artifact: the agent’s own reading becomes part of its state, so a defense must both compete explanations at the moment of reading and keep revised rules in force afterwards. CERS implements this pair with a competing-explanation chain over visible chart evidence, in-chart applicability verification, and a persistent rule state revised only through KEEP/REVISE/UNRESOLVED decisions with recorded evidence and scope. The static panel shows a within-run robust-accuracy gain that replicates across seeds and transfers to an 8B agent through the decision-oriented variant, while the failure analysis shows that interface contracts co-determine outcomes as strongly as prompt content. The stateful deployment shows the complement: under a fail-closed protocol the defense arm never submits a wrong answer and its residual errors become legible abstentions with recorded reasons. We release decision logs, rule states, and re-derivation traces so future work can measure which explanation an agent acted on and whether it survived contact with the chart.

Limitations

This work studies defensive scaffolding around vision–language agents, and its limitations follow from the same design choices that produce the results.

Scale and models. The static panel uses one 27B open-weight VLM and one 8B open-weight VLM under fixed decoding; the online deployment uses the same 27B model under two interface regimes

and one frontier model accessed through a gateway. We do not claim that the reported gains generalize to proprietary or future models, to other agent frameworks, or to tasks outside the four covered domains; the 8B transfer already shows that variant choice interacts with actor capability in ways we do not fully predict.

What the labels measure. Robust accuracy is computed against the original option labels of the benchmark, and our agreement checks verify that submitted options and rule states match the recorded logs. Label agreement is not semantic verification: a run whose recorded reasoning matches its outcome may still rest on a misreading that the frozen labels do not separate, and conversely a correct submission may be produced for partially wrong reasons. The rule states we analyze are the agent’s own declared rules, not independently validated facts.

Selection effects and variance. The paired gains are within-run comparisons on a fixed task set; the drop of direct selection from 81 to 73 of 140 under an identical configuration quantifies sampling variance, and we therefore report McNemar tests and exact confidence intervals on paired decisions rather than cross-run rate differences. The 24-task development subset was used to calibrate configuration emphasis and is scored separately throughout; still, all configurations were designed by the authors after inspecting aggregate failure patterns, so the panel should be read as a controlled comparison rather than a held-out evaluation.

Interface dependence. The online results are conditional on the strict submission protocol. Its semantic contract—exact public-leaf values, bounded summaries, count-coupled responses—cannot be expressed in a response format, and under earlier actor systems the same machinery died in engineering failures rather than abstentions. The deployment therefore demonstrates the behavior of CERS under one operating point (frontier actor, direct gateway, strict typed interface), not a interface-robust defense; the negative environment-domain results of the static panel and the one-directional online paired effect are reported as found.

Runtime footprint. Verification multiplies requests by roughly $2.6\times$ over direct selection and the online arm operates within a fixed per-unit budget (1 re-verification round, 8 actor calls); agents

deployed under tighter latency or cost envelopes may not accommodate this overhead.

Ethics and dual use. The defense direction of this work is protective, but the same observation—that faithful chart pixels admit misleading encodings that change executed actions—can inform adversarial chart design; we therefore release instances only through the benchmark’s own access process and avoid publishing new manipulation techniques.

Reproducibility boundary. The companion benchmark is under review and its instances are not yet public; until release, the results here cannot be independently recomputed from the logs alone, although all decision logs, prompts, and rule states referenced in this paper are preserved verbatim in the submission’s supplementary material.

References

- Jason Alexander, Priyal Nanda, Kai-Cheng Yang, and Ali Sarvghad. 2024. [Can gpt-4 models detect misleading visualizations?](#) In *2024 IEEE Visualization and Visual Analytics (VIS)*.
- Shubham Bharti, Shiyun Cheng, Jihyun Rho, Jianrui Zhang, Mu Cai, Yong Jae Lee, Martina Rau, and Xiaojin Zhu. 2024. [Chartom: A visual theory-of-mind benchmark for llms on misleading charts](#). *Preprint*, arXiv:2408.14419.
- Graziano Blasilli and Marco Angelini. 2026. [True \(vis\) lies: Analyzing how generative ai recognizes intentionality, rhetoric, and misleadingness in visualization lies](#). In *arXiv (Cornell University)*. Cornell University.
- Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2024a. [Struq: Defending against prompt injection with structured queries](#). *Preprint*, arXiv:2402.06363.
- Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. 2024b. [Secalign: Defending against prompt injection with preference optimization](#).
- Zixin Chen, Sicheng Song, Kashun Shum, Yanna Lin, Rui Sheng, Weiqi Wang, and Huamin Qu. 2025. [Unmasking deceptive visuals: Benchmarking multimodal large language models on misleading chart question answering](#).
- Amit Kumar Das, Mohammad Tarun, and Klaus Mueller. 2025. [Charts-of-thought: Enhancing llm visualization literacy through structured data extraction](#). In *IEEE Transactions on Visualization and Computer Graphics (TVCG)*, *PrePrints 5555*, pp. 1-11, 2025.

- Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. [Improving factuality and reasoning in language models through multiagent debate](#). arXiv:2305.14325. *Preprint*, arXiv:2305.14325.
- Ivan Evtimov, Arman Zharmagambetov, Aaron Grattafiori, Chuan Guo, and Kamalika Chaudhuri. 2025. [Wasp: Benchmarking web agent security against prompt injection attacks](#). *Preprint*, arXiv:2504.18575.
- Xueyu Hu, Tao Xiong, Biao Yi, Zishu Wei, Ruixuan Xiao, Yurun Chen, Jiasheng Ye, Meiling Tao, Xianguan Zhou, Ziyu Zhao, Yuhuai Li, Shengze Xu, Shenzhi Wang, Xinchun Xu, Shuofei Qiao, Zhaokai Wang, Kun Kuang, Tieyong Zeng, Liang Wang, and 10 others. 2025. [Os agents: A survey on mllm-based agents for computer, phone and browser use](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7436–7465, Vienna, Austria. Association for Computational Linguistics.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2023. [Large language models cannot self-correct reasoning yet](#). *Preprint*, arXiv:2310.01798.
- Sam Johnson, Viet Pham, and Thai Le. 2025. [Manipulating llm web agents with indirect prompt injection attack via html accessibility tree](#).
- Rachneet Kaur, Nishan Srishankar, Zhen Zeng, and Sumitra Ganesh. 2026. [Chartagent: A multimodal agent for visually grounded reasoning in complex chart question answering](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18488–18555, San Diego, California, United States. Association for Computational Linguistics.
- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. 2024. [Visualwebarena: Evaluating multimodal agents on realistic visual web tasks](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 881–905, Bangkok, Thailand. Association for Computational Linguistics.
- Yin Li. 2025. [Decomposing llm self-correction: The accuracy-correction paradox and error depth hypothesis](#). arXiv:2601.00828. *Preprint*, arXiv:2601.00828.
- Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. 2024. [Encouraging divergent thinking in large language models through multi-agent debate](#). pages 17889–17904.
- Maxim Lisnic, Cole Polychronis, Alexander Lex, and Marina Kogan. 2023. [Misleading beyond visual tricks: How people actually lie with charts](#). pages 1–21.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. [Self-refine: Iterative refinement with self-feedback](#). arXiv:2303.17651. *Preprint*, arXiv:2303.17651.
- Ridwan Mahbub, Mohammed Saidul Islam, Md Tahmid Rahman Laskar, Mizanur Rahman, Mir Tafseer Nayeem, and Enamul Hoque. 2025. [The perils of chart deception: How misleading visualizations affect vision-language models](#). In *2025 IEEE Visualization and Visual Analytics (VIS)*.
- Ridwan Mahbub, Mohammed Saidul Islam, Md Tahmid Rahman Laskar, Mizanur Rahman, Mir Tafseer Nayeem, and Enamul Hoque. 2026. [Chart deception in vision-language models: From vulnerability to mitigation](#). *Preprint*, arXiv:2607.22600.
- Q McNEMAR. 1947. [Note on the sampling error of the difference between correlated proportions or percentages](#). In *Psychometrika*.
- Khandaker Abrar Nadib, Marina Kogan, Alexander Lex, and Maxim Lisnic. 2026. [Guardrail selection in line charts to contextualize persuasive visualizations](#). In *Computer Graphics Forum (EuroVis)*, 45(3), 2026.
- Swaroop Panda. 2026. [Making ai agents evaluate misleading charts without nudging](#).
- Christian Seto, Jacqueline Nguyen, Jiayi Hong, and Ross Maciejewski. 2026. [Llms have visualization literacy: Now what? experiments exploring llm visualization evaluation capabilities](#). In *arXiv (Cornell University)*. Cornell University.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. [Reflexion: Language agents with verbal reinforcement learning](#). *Preprint*, arXiv:2303.11366.
- Jonathan Tonglet, Tinne Tuytelaars, Marie-Francine Moens, and Iryna Gurevych. 2026. [Protecting multimodal large language models against misleading visualizations](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8329–8349, San Diego, California, United States. Association for Computational Linguistics.
- Jonathan Tonglet, Jan Zimny, Tinne Tuytelaars, and Iryna Gurevych. 2025. [Is this chart lying to me? automating the detection of misleading visualizations](#). arXiv:2508.21675. *Preprint*, arXiv:2508.21675.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yuan Yang, Jingsen Zhang, Zhiyuan Chen, Jikai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2024. [A survey on large language model based autonomous agents](#). *Frontiers of Computer Science*, 18(6).

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. [Self-consistency improves chain of thought reasoning in language models](#).

Zhun Wang, Vincent Siu, Zhe Ye, Tianneng Shi, Yuzhou Nie, Xuandong Zhao, Chenguang Wang, Wenbo Guo, and Dawn Song. 2025. [Agentvigil: Automatic black-box red-teaming for indirect prompt injection against llm agents](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 23159–23172, Suzhou, China. Association for Computational Linguistics.

Chen Henry Wu, Rishi Shah, Jing Yu Koh, Ruslan Salakhutdinov, Daniel Fried, and Aditi Raghunathan. 2024. [Dissecting adversarial robustness of multi-modal lm agents](#). *Preprint*, arXiv:2406.12814.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. [Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents](#). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, Bangkok, Thailand. Association for Computational Linguistics.

Yanjie Zhang, Yafei Li, Rui Sheng, Zixin Chen, Yanna Lin, Huamin Qu, Lei Chen, and Yushi Sun. 2026. [Navigating the mirage: A dual-path agentic framework for robust misleading chart question answering](#). In *arXiv (Cornell University)*. Cornell University.

Wei Zhao, Zhe Li, Peixin Zhang, and Jun Sun. 2026. [Clawguard: A runtime security framework for tool-augmented llm agents against indirect prompt injection](#). *Preprint*, arXiv:2604.11790.

A Execution Loop and Worked Example

Algorithm 1 summarizes one execution of CERS under the candidate budget K and the review budget L for a single verification node.

A.1 Rule Merging by Object and Scope

“Darker means higher risk” and “a deeper color stands for a more severe risk” map to one rule when object and scope coincide, whereas comparing Illinois with Kansas and Illinois with Delaware under the same legend are two comparisons that shared rules do not merge.

A.2 Update Objects and Chain Exclusion

When a chain is challenged, the method distinguishes three objects of update: an *observation error* (a misread number, wrong entity binding, or wrong range) revises the affected O and re-verifies dependent interpretations, never charging unreliable observations to B ; an *inapplicable rule*—whose observation stands while its observation-to-semantics mapping disagrees with the current

chart—is revoked or scope-narrowed; and *insufficient derivation* leaves both supported while the task conclusion remains unproven, so the module adds the missing comparison or keeps an undetermined status, as when a higher candidate refutes “the current candidate is highest” without confirming the alternative is globally highest.

Conflicting conclusions between candidate chains establish only that a conflict exists. A chain is excluded only when chart-supported evidence negates its observation, its rule applicability, or its derivation conditions.

A.3 A Worked Example of Revision and Cross-Step Reuse

We illustrate the mechanism on the chart semantics of task pub013 from our execution data, which asks the agent to choose the highest-risk record among Illinois, Kansas, and Delaware; the case material records that the map legend assigns lighter yellow to higher risk and deep red to lower risk. The walkthrough below demonstrates the method and does not add experimental results. Suppose the current proposal selects Kansas under

$$\pi_0 : \underbrace{\text{Kansas is darker}}_{O_0} \wedge \underbrace{\text{darker means higher risk}}_{B_0} \Rightarrow C_q(\text{KS}). \quad (8)$$

The competing module asks why a darker color should mean a higher risk; the model generates legend-dependent interpretations on the same complete map and compares the legend with the default color convention. If verification confirms that the chart does not support B_0 , the rule loses applicability on this map and the state records the original interpretation, the revision reason that the convention contradicts the current legend definition, the current interpretation that candidate colors are read through the verified legend mapping, the scope of the current map and risk metric, and the condition for further updates, namely a different encoding or an effective refutation of the legend reading. The agent then re-compares the candidates with B_1 and the observed colors and completes the judgment the public task requires; the concrete option is an output of this reasoning, while the rule state keeps the interpretation basis of the current map. When the agent enters later pages or re-visits the chart-dependent choice, what it reads is that rule and its revision reason: a new explanation that merely repeats “deep red usually means more danger” must answer the recorded legend evidence and cannot

restore B_0 directly, while a re-reading that finds the earlier legend verification mistaken is allowed to revise again. The process records three distinct things: what changed in the interpretation rules, whether the current action must change, and which rule version later steps actually used.

B Additional Results and Boundary Statements

B.1 Matched-Configuration Seed Replication

The revision experiments rerun direct selection and the full configuration under three request seeds (12345, 22345, 32345) on a single-GPU deployment whose engine configuration matches the frozen service except for batch admission (`max_num_seqs=8` instead of 1), so all comparisons reported for these runs are internal to that configuration. The three plain runs also provide the three samples for the self-consistency baseline at no additional requests, with majority vote among non-null selections and ties recorded as abstentions. The tabular-answering baseline re-requests each task in two stages under the same engine: a transcription request that renders, from the image, a compact markdown table limited to the series and x positions the public task’s comparison involves (at most 24 rows), and a decision request answered from the table alone, so the decision stage never sees the image. The clean-chart control runs plain direct selection on the paired clean charts and is scored against the original labels, the protocol established by the companion benchmark’s online evaluation.

B.2 Development Subset and Stratified Results

The 24-task development subset is scored separately in every round. In the original round the full configuration V5 gains +17 paired decisions on the remaining 116 tasks and +1 on the development subset; the same decomposition holds for the 8B round (+10 splits as +9 and +1). The environment domain is the only one in which V5 loses against direct selection in both model rounds (−5 in the 27B replication, −9 in the 8B round). At the family level, subset-extrapolation tasks contribute +18 (original round) and +11 (8B round) for V5; dual-axis and non-linear-axis families are mildly negative for V5 in the second model round (−5 and −6), and bar-height tasks are near-neutral (−3 and −2).

B.3 Interface-Failure Accounting

Interface failures are runs in which the agent never produces a schema-valid selection. They are zero for Direct, V4, V5, V6, and V7 on the 27B model (all three rounds), and 6, 8, and 51 for V3 in the original, replication, and 8B rounds respectively. V3’s interface failures are not discarded: they remain in the denominator, which is why its large error-to-target correction counts on the 27B model (30 forward against 6 reverse, exact McNemar $p = 0.0001$) reduce to a net +2. The 8B actor’s failure mode is dominated by responses that violate the typed-record contract of the verification pass rather than by timeouts or truncations.

B.4 Online Deployment Details

Each online unit executes one task on one chart variant under one system inside a sealed environment: public inputs are mounted read-only, rule hooks intercept chart evidence entering the context, and the unit ends in exactly one of the recorded terminal statuses (submitted, abstention with reason, re-verification budget exhausted, or engineering failure). The strict interface (`typed_scalars.v2`) permits at most eight actor calls and one re-verification round per unit at temperature zero. The three actor systems differ only below the interface: a local 27B model under strict response formats (Batch A), the same model behind a schema-guided proxy mirroring the sealed structural rules (Batch B), and the frontier model reached directly through the gateway (Batch C and its stability rerun). Across the two frontier-model batches the defense arm read its persistent rule state 48 and 46 times across its 24 units, never cooccurred with a wrong submission, and every per-unit persistence-causality flag reads `not_established`; the ordinary arm produced exactly six wrong submissions, all chart-induced, all under the 27B actor on misleading charts, and none under the frontier actor.

B.5 Boundary Statements

Three boundary statements delimit the claims. First, the per-unit `causal_persistence` effect is recorded as *not established* in every online unit: the harness shows that revised rules were re-read and that outcomes changed between arms, but a single-unit causal attribution of outcome change to persistence is not identified by this design. Second, the stability rerun’s unit-level agreement (37/48) bounds the reliability of any single-batch online es-

timate; disaggregated four-way cells shift by at most two units, and no flip ever crosses into a wrong submission. Third, the online deployment’s paired effect is one-directional: the defense arm converts ordinary decisions into abstentions (10 and 13 per batch) and never the reverse, so the deployment should be read as a safe-fail characterization of the defense under the strict protocol, not as an accuracy improvement result.

Algorithm 1 CERS execution loop

- 1: inputs: task q ; base agent $\mathcal{F}_\theta$ and the original action interface; chain budget K ; review budget L .
 - 2: initialize rule state $M \leftarrow \emptyset$; execution history $H \leftarrow \emptyset$.
 - 3: **for** each step t **do**
 - 4: receive observation o_t ; update chart references and interpretation context c_t .
 - 5: read applicable rules, revision evidence, and disputes from M .
 - 6: $(\tilde{a}_t, \tilde{\pi}_t) \leftarrow \mathcal{F}_\theta(q, o_t, H, \mathcal{R}_t)$.
 - 7: **if** a trigger holds: first chart-dependent judgment, interpretation conflict, or applicability change **then**
 - 8: expand the current explanation into $O \wedge B \Rightarrow C_q(\tilde{a}_t)$.
 - 9: generate at most K substantially different competing chains on the same complete chart.
 - 10: normalize candidates; merge repeated chains and shared rules.
 - 11: verify observations, rule applicability, and derivation sufficiency.
 - 12: apply adoption, revocation, replacement, or scope adjustment by the evidence.
 - 13: re-derive the current action from the updated state; within L , resolve remaining key disputes.
 - 14: **end if**
 - 15: **if** the proposal reuses a revoked rule without new evidence, return the revision record and require re-derivation; with an effective new challenge, re-verify the rule.
 - 16: with KEEP or REVISE, execute the approved action through the original interface; with exhausted budget, record UNRESOLVED and stop the decision.
 - 17: append the true observation, proposal, rule versions, action, and feedback to H .
 - 18: **end for**
 - 19: save the task’s rule revisions and execution trajectory; close the task’s rule scope.
-